

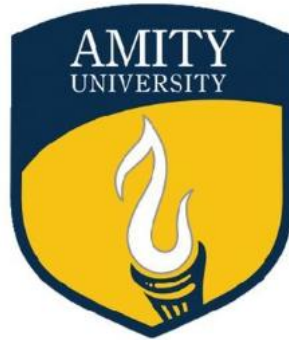
Report Paper

In-House NTCC 2026

Automated Knowledge Graph using NLP

Submitted to

Amity University Noida, Uttar Pradesh



In partial fulfillment of the requirements for the award of degree
of Bachelor of Technology

In

Computer Science and Engineering By

Nevaeh Singh(A2305224144)

Under the guidance of

Dr. Sumit Kumar

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
AMITY SCHOOL OF ENGINEERING AND TECHNOLOGY AMITY
UNIVERSITY NOIDA, UTTAR PRADESH

Batch:2024-2028

DECLARATION

I, Nevaeh Singh students of B.Tech(Computer Science Engineering) hereby declare that the In-House NTCC Report titled:- “Automated Knowledge Graph using NLP” which is submitted by us to Amity School of Engineering and Technology, Amity University Noida, Uttar Pradesh, in partial fulfilment of requirement for the award of the degree of Bachelor of Technology (Computer Science Engineering), has not been previously formed the basis for the award of any degree, diploma or other similar title or recognition.

Nevaeh Singh
(A2305224144)

ACKNOWLEDGEMENT

It is our privilege to express our deep sense of gratitude to those entire faculty Members who helped us in the completion of the Internship report, especially our guide Dr. Sumit Kumar who was always there at hour of need. My special thanks to all other faculty members, Batch mates & Seniors of Amity School of Engineering and Technology (ASET) Amity University, Uttar Pradesh for helping me in the completion of Internship work and its report submission.
Above all, We pay our reverence to the almighty GOD.

Nevaeh Singh
(A2305224144)

Index

S.No.	Table of Contents	Page No.
	Abstract	1
1.	Introduction	2-3
1.1	Background	2
1.2	Problem Statement	3
1.3	Objective	3
2.	Literature review	4-12
2.1	Overview	4
2.2	Review of existing papers	4-7
2.3	Summary of LR	7-12
2.4	Research Gap	12
3.	Methodology	13- 22
3.1	Overview and system architecture	13
3.2	Document Ingestion Module	14
3.3	Text Preprocessing	16

3.4	Named Entity Recognition	17
3.5	Relationship Extraction	19
3.6	Neo4j Graph Database and Knowledge Graph Visualization	21
4	Result and Analysis	23-30
4.1	Software configuration	23
4.2	Application Interface	24
4.3	Evaluation using different documents	24
5	Applications	30
6	Issues and Challenges	31
7	Conclusion	32
8	Future scope	33
9	References	34

Abstract

With the rapid growth of digital information, manually extracting insights from a vast amount of unstructured textual data is time-consuming, tedious and inefficient. The aim of this project is to automatically transform unstructured text from PDF, DOCX and TXT into a structured knowledge graph using Natural Language Processing techniques and a graph database. The system extracts textual content applies preprocessing pipeline and utilizes Named Entity Recognition to identify key entities such as persons, organization, location, date and money. It then performs relationship extraction to map connections as Subject-Relation-Object (SRO) triples which are stored in the Neo4j graph database. To facilitate user interaction, a Streamlit web application was developed, integrating PyVis for interactive, real-time graph visualization and semantic exploration. The system was evaluated across diverse document types including research papers, annual reports and articles. Experimental results demonstrate that the pipeline successfully automates knowledge graph generation significantly reducing manual analysis effort. This scalable approach offers high utility for domains like business intelligence, semantic searches and knowledge management.

Chapter 1

Introduction

In the current digital age, there is a huge amount of information being produced every day through research papers, reports, articles, books, and web documents. The majority of such information is present in an unstructured format, which makes it challenging to manage and analyse. Manually extracting important concepts and relations from these documents is a time-consuming, laborious task, which also becomes quite difficult when dealing with a large amount of information. In this context, intelligent systems that can automatically transform unstructured text into structured knowledge are becoming extremely necessary.

Natural Language Processing (NLP) is a domain within Artificial Intelligence that provides computers with the ability to understand and extract information from human language. Through the combination of NLP with Knowledge Graph, we can represent our textual information in the form of connected entities and relations.

The Automated Knowledge Graph Builder Using NLP is a solution aimed at automating the task of knowledge extraction from documents. The system takes input in the form of PDF, DOCX, and TXT files, pre-processes the extracted text, identifies significant entities by means of Named Entity Recognition (NER), analyses the relations between these entities, and generates a graph stored in the Neo4j graph database. The obtained knowledge graph can be analysed via an interactive visualization interface that allows users to make sense of complicated data easily.

1.1 Background

Given the increasing use of digital texts in businesses today, companies are encountering serious issues when trying to extract information from massive amounts of unstructured text. Information is stored in traditional databases in tabular form but cannot effectively represent semantic connections between various entities contained in these texts. In turn, this means that the analysis of massive collections of documents involves manual work. With the help of Knowledge Graphs, information can be represented as nodes (entities) and edges (relations). This allows one to intelligently query, reason about semantics, and visualize interrelated knowledge. With modern technologies of Natural Language Processing such as tokenizing, text pre-processing, Named Entity Recognition (NER), and relations extraction, one can effectively recognize important entities and their relationships in unstructured texts.

New developments in open-source platforms like Python, spaCy, Neo4j, Streamlit, and PyVis have facilitated the creation of intelligent systems which are capable of automatically creating knowledge graphs out of text documents. These intelligent systems are being used more often in educational institutions, healthcare facilities, scientific research, business intelligence, recommendation systems, search engines, and Q&A systems. NLP, along with graph databases, has become a reliable way to turn text data into knowledge.

1.2 Problem Statement

A vast amount of valuable data resides in unstructured documents such as research papers, technical reports, articles, and books. It is difficult and time-consuming to manually retrieve useful information from such documents and is often inconsistent. The conventional techniques of information retrieval work mostly through keyword searches and do not consider the relationships among entities in a document.

The available document processing systems concentrate solely on text extraction or entity recognition but do not offer a solution to generate a knowledge graph and its visualization. Such limitations restrict user understanding of relations among extracted concepts and their use for further processing.

Thus, an automated system capable of processing various types of documents, extracting entities and relationships using NLP methods, storing retrieved information into a graph database, and generating a knowledge graph would serve to be extremely helpful.

1.3 Objectives

- To develop a system capable of accepting PDF, DOCX and TXT documents.
- To extract textual content from uploaded documents accurately.
- To preprocess the extracted text by removing unnecessary words, symbols, and duplicate information.
- To improve information retrieval and semantic understanding of large collections of textual documents.
- To reduce manual effort in knowledge extraction and enable efficient document analysis.

Chapter 2

Literature Review

2.1 Overview

The Automated Knowledge Graph Builder Using NLP is an example of a project that utilizes the modern approaches to Natural Language Processing to transform text documents into knowledge graphs. Namely, this project automatically identifies both entities and relations between them within unstructured texts such as scientific papers, reports, and articles. Afterward, the acquired information is organized into graphs where entities are presented as nodes while their relations are presented as edges. This allows users to get a better visual representation of the complex information.

This automated process includes three main components—state-of-the-art NLP algorithms, a graph database, and a web application—that cooperate in order to acquire knowledge from unstructured documents. The processes like text preprocessing, Named Entity Recognition (NER), relation extraction, and graph construction are performed, while the output knowledge graph is stored in the Neo4j graph database.

Knowledge graphs have become a critical component of intelligent systems by creating a structure that enables semantic-based search, question answering, recommendation services, and decision support applications. Nowadays, the trend in research shows increasing collaboration between knowledge graphs, NLP, and deep learning technologies that results in significant progress in information extraction, semantic understanding, and reasoning.

The main goal of this research is enabling the automatic extraction of knowledge from large amounts of text-based resources. With the help of NLP and graph databases, this approach provides a solution for transforming simple text into a usable representation of knowledge that can be applied in various fields like education, medicine, and science.

2.2 Review of existing papers

Knowledge graphs were introduced in 2021 in studies carried out by Aidan Hogan and his colleagues. It describes the significance of knowledge graphs in efficiently integrating, managing, and extracting massive heterogeneous data. It is significant in both academic and industrial environments, including areas of Artificial Intelligence, Machine Learning, Semantic Web Technologies, and Graph Neural Networks. Ontologies, graph embeddings, and reasoning are a few factors that were emphasized in order to enhance knowledge representation and inference. In conclusion of their paper, the authors indicate that knowledge graphs have evolved into an essential technology for intelligent systems and list some areas for further research. [1]

The survey summarizes a review of the use of knowledge graphs throughout the last decade. The paper stresses that the use of knowledge graphs is an effective way to

improve semantic representation and context comprehension of NLP systems. Furthermore, new tendencies like graph embeddings, transformer-based models, and graph-enhanced language models are discussed. The conclusion made by the authors is that using knowledge graphs has become an important trend in Artificial Intelligence research. This work will further evolve in knowledge extraction and reasoning systems automation. The authors conduct an analysis of 507 research papers and identify major NLP tasks using knowledge graphs, including information extraction, question answering, semantic search, and text understanding. [2]

This article discusses the comparative analysis of several relation extraction methods used to create a knowledge graph automatically using Natural Language Processing. The article compares the efficiency of diverse algorithms in practical cases. The algorithms analysed include REBEL, KnowGL, KeyBERT, and GPT-4. The authors developed an annotated dataset based on news articles and performed experiments analysing the methods by comparing their precision, recall, F1-measure, clustering coefficient, and using qualitative analysis. The results prove that GPT-based large language models have better performance than other relation extraction algorithms in creating knowledge graph triples. At the same time, algorithms like REBEL and KnowGL excel at providing additional external semantic content and increasing graph connectivity. Overall, automated knowledge graph extraction appears promising but challenging. [3]

This paper represents an efficient relationship extraction approach for building domain-specific knowledge graphs, integrating information from different data sources. This approach utilizes classification systems, co-occurrence, and semantic analysis to extract hierarchical relationships, whereas non-hierarchical relationships are extracted by a convolutional RNN with an attention mechanism and an improved cross-entropy loss function. This allows the model to learn better relationships and improve the overall quality of the knowledge graph. Experimental results show that the proposed technique works better than other relation extraction techniques. Through the application of the technique to a dataset related to the food domain, the researchers were able to create a knowledge graph. It shows that the technique is effective and feasible. According to the findings of the study, the use of various extraction methods for different kinds of data creates a more complete knowledge graph. [4]

The method REBEL utilizes an end-to-end model for relation extraction by casting it into a sequence-to-sequence problem for text generation, contrary to the existing multi-step pipeline approach. Leveraging the BART transformer-based model, REBEL generates relation triples directly from unstructured text without resorting to separate entity recognition and relation classification phases. To improve its capacity, the model has been trained using a large-scale corpus from Wikipedia and Wikidata and therefore is able to detect more than 200 types of relations. According to experimental findings, the model performs well on several benchmark corpora at the state-of-the-art level and with less fine-tuning compared to previous approaches. The simplicity of the model and its generalization ability make it a powerful method for constructing knowledge graphs and relation extraction. [5]

The paper introduces Grapher, a comprehensive multi-stage solution to construct KGs in an end-to-end manner from text inputs. Contrary to earlier work that linearizes the

whole graph into a single sequence, Grapher separates the problem into two parts: (i) node generation, which extracts entities based on a pre-trained language model (T5), and (ii) edge generation, in which relationships between these nodes (entities) are predicted. The main advantage is the avoidance of graph linearization redundancy and allowing end-to-end learning without using any other NLP pipeline such as entity or relation extraction tools. Grapher is tested on three benchmark datasets, namely, WebNLG 2020, TEKGEM, and NYT. From the results, it can be observed that the method of Grapher is on par with the WebNLG dataset but outperforms previous methods on the larger TEKGEM and NYT datasets. In addition to that, the authors have developed sparse matrix training methods to balance the problem of imbalance between existent and non-existent edges. Finally, the authors conclude that separating node and edge generation is an efficient strategy for KG construction. [6]

This chapter presents the notion of Knowledge Graphs (KGs) and provides an overview of their increasing relevance to modern intelligent systems. In particular, the authors focus on the role of knowledge graphs as sources of structured data and facts that underlie different applications, including search engines and personal assistants. At the same time, the authors point out that, although knowledge graphs are extensively discussed in the literature, there is currently no unified definition for this notion. Hence, to provide conceptual clarification for the emergence of knowledge graphs, the authors discuss their evolution and characteristics. The writers believe that the knowledge graph provides a good framework for organizing real-world knowledge into data that can be easily integrated and reasoned over. This forms a basis for more complex applications of Artificial Intelligence technology. [7]

The paper describes a methodology using deep learning technology to carry out Named Entity Recognition (NER) and build knowledge graphs based on geological hazard-related literature. The research group created a deep multi-branch BiGRU-CRF architecture that can perform automatic entity extraction, including geographical locations, methodologies, and data. To facilitate such an endeavor, a pattern-based method of corpus creation is proposed in order to provide features generated via system learning and thereby diminish the need for manual feature creation. According to the experiments, the proposed deep multi-branch BiGRU-CRF architecture performs better than existing NER solutions in terms of precision, recall, and F1-score metrics and successfully created a large-scale geological hazard knowledge graph including 34,457 entity nodes and 84,561 relations. [8]

The current paper introduces Plumber, an information extraction framework that aims to improve knowledge graph completion by means of unstructured data. Contrary to a static pipeline approach, Plumber employs 40 reusable NLP modules to complete tasks related to triple extraction, entity linking, and relation linking. An approach based on RoBERTa classification allows the system to automatically choose the best pipeline for knowledge graph triple extraction. The experimental results clearly indicate that Plumber generates better results than static pipelines and state-of-the-art baselines on DBpedia, Wikidata, and ORKG. From these results, we can safely draw the conclusion that dynamically choosing information pipelines based on the text input greatly enhances the performance of the knowledge graph generation process and provides a scalable way of using different NLP models. [9]

This survey offers a detailed analysis of knowledge graphs and their significance in the fields of Artificial Intelligence and big data applications. The paper discusses the way knowledge graphs are used to capture real-world objects and the relations between them to make machine processing of the data possible. The authors discuss knowledge graph opportunities in AI, including their usage in the creation of recommendation systems, question answering, and information retrieval. In addition, applications of knowledge graphs in areas such as education, scientific research, social networking, and healthcare are discussed. The paper also analyses the main technical limitations that currently prevent progress in the field of knowledge graphs, including knowledge acquisition, graph embeddings, graph completion, knowledge fusion, and knowledge reasoning. The authors note that despite the significant improvement in intelligent applications and decision-making due to knowledge graphs, there is a need for further research concerning issues such as scalability and data integration. [10]

This paper presents the idea of a deep learning model known as PRE-Span, which aims at extracting entities and relations together for knowledge graph construction. As opposed to previous pipeline models that extract entities and relations sequentially, PRE-Span uses two separate components that work simultaneously to minimize errors. PRE-Span is developed based on BERT with Multi-Head Self-Attention, Bi-LSTM, and a feed-forward network, helping to produce relational triples in a single step. It has been proved experimentally that PRE-Span performs better than many other joint extraction models on the basis of precision, recall, and F1-score. Moreover, PRE-Span saves time during the training phase owing to its parallel design. According to the results of this study, PRE-Span proves to be a good technique for relation extraction and knowledge graph construction. However, it faces problems regarding nested entities and long-distance dependencies. [11]

2.3 Summary of Literature review

Table 2.1: Comparative Study of Existing Research

Title	Authors	Conclusion	Research Gap
Knowledge Graphs [1]	Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia D'Amato, Gerard de Melo, Claudio Gutierrez	This paper is an elaborate discussion on knowledge graphs, which includes the model, querying, reasoning, and application of knowledge graphs. The conclusion drawn from this analysis is that knowledge graphs are important in AI in that they allow effective knowledge representation and decision-making.	This paper is an overview, and does not introduce any new implementations or experimentation. There is still a need for more research to enhance knowledge graph construction, automatic extraction and reasoning from dynamic data.

A Decade of Knowledge Graphs in Natural Language Processing [2]	Phillip Schneider, Tim Schopf, Juraj Vladika, Mikhail Galkin, Elena Simperl, and Florian Matthes	The survey summarizes the last decade of research related to knowledge graphs used in Natural Language Processing through the examination of 507 papers. The survey categorizes major tasks, points out major trends in the field, and draws attention to the increasing use of knowledge graphs in areas like information extraction, question answering, semantic search, and language modeling. Knowledge graphs have been found to be an important element of NLP, but the survey stresses the necessity of additional research.	A lack of benchmark datasets, empirical evaluations, and secondary literature review has been identified by the survey. Furthermore, it stresses the importance of real-world applications, reuse of resources, and extensive evaluation in other domains for progress in NLP knowledge graph research.
A Relationship Extraction Method for Domain Knowledge Graph Construction [3]	Haoze Yu, Haisheng Li, Dianhui Mao, Qiang Cai	In this research paper, we present an approach for relation extraction towards building domain knowledge graphs with the help of structured, semi-structured, and unstructured data. We have applied a convolution residual network along with an enhanced version of the cross-entropy loss function to make the process of relation extraction more precise. The results obtained using a knowledge graph in the domain of food have shown improvements in the process.	Limited to a specific domain and requires validation on larger, more diverse datasets.
REBEL: Relation Extraction by End-to-end Language Generation [4]	Pere-Lluís Hugué Cabot, Roberto Navigli	REBEL presents a framework that performs relation extraction using a sequence-to-sequence architecture that takes in raw text input and outputs relation triples using a BART based model. Through the removal of the entity recognition and relation extraction steps, REBEL not only minimizes errors in the pipeline but also attains state-of-the-art results on several benchmark datasets.	Needs further evaluation on specialized domains and multilingual datasets.
From Text to Knowledge	RoosM. Bakker, Daan L. DiScala	This paper analyzes various relation extraction algorithms for automatic construction of	Requires larger datasets and evaluation across

Graph: Comparing Relation Extraction Methods in a Practical Context [5]		knowledge graph in a realistic setting. With the help of the manually labeled data, the paper assesses the traditional models, REBEL, KnowGL and GPT based algorithms. The results indicate that GPT models have proven to be most effective, whereas the models like REBEL and KnowGL have added external knowledge in the graph.	more application domains.
Knowledge Graph Generation from Text [6]	Igor Melnyk, Pierre Dognin, Payel Das	The authors come up with a novel framework which involves creating nodes in graphs followed by predicting edges without linearizing the graphs. This novel method performs as good as, or even better than, other methods on several benchmark datasets.	Scalability for larger graphs and multilingual support remain open challenges.
Introduction: What Is a Knowledge Graph? [7]	Dieter Fensel et al.	This chapter describes the idea, rationale, and development of knowledge graphs. It addresses how knowledge graphs can be used to represent structured knowledge, provide semantic understanding, and develop intelligent systems like search engines and virtual assistants. Additionally, it sets up the fundamental ideas for future knowledge graph studies.	As a conceptual chapter, it lacks implementation and experimental validation.
Deep Learning-Based Named Entity Recognition and Knowledge Graph Construction for Geological Hazards [8]	Runyu Fan, Lizhe Wang, Jining Yan, Weijing Song, Yingqian Zhu, Xiaodao Chen	A multi-branch deep BiGRU-CRF model is provided for performing named entity recognition on geological hazards documents. In combination with the corpus construction technique based on patterns, the technique reaches high precision and constructs a large domain knowledge graph, which proves that deep learning techniques can be utilized in domain-specific information extraction.	Limited to geological data and requires broader cross-domain validation.
Information	Mohamad Yaser Jaradeh,	The Plumber approach is presented as a flexible architecture	Can be extended with additional NLP

Extraction Pipelines for Knowledge Graphs [9]	Kuldeep Singh, Markus Stocker, Andreas Both, Sören Auer	that builds an optimal information extraction pipeline for each input text. With the help of reusable NLP components and pipelines selection based on the transformer classifier, Plumber helps to improve knowledge graph completion.	components and real-world evaluations.
Knowledge Graphs: Opportunities and Challenges [10]	Ciyuan Peng, Feng Xia, Mehdi Naseriparsa, Francesco Osborne	The survey provides an overview of the potential uses, application areas, and technical issues of knowledge graphs for artificial intelligence. The topics covered include knowledge acquisition, embedding, completion, reasoning, and applications in different fields, giving a general overview of the state of knowledge graph research today and tomorrow.	More practical implementations and scalable solutions are needed.
A Parallel Model for Jointly Extracting Entities and Relations [11]	Zuqin Chen, Yujie Zheng, Jike Ge, Wencheng Yu, Zining Wang	The PRE-Span technique employs parallel architecture that allows the simultaneous recognition of both entities and relations and reduces errors associated with pipelined systems. The effectiveness of PRE-Span is evident from experimental results, demonstrating increased precision and efficiency on several benchmark datasets.	Handling nested entities and long-distance relations remains challenging.

2.4: Research Gap

Existing research shows that much progress has been achieved in the construction of knowledge graphs through methods like Named Entity Recognition (NER), relation extraction, end-to-end deep learning, and dynamic information extraction systems. Nevertheless, most of the proposed solutions have been developed and tested on particular datasets or domain-specific tasks like geological hazards, food knowledge, and benchmark datasets, thus hindering their ability to generalize to other domains.

Moreover, many of the existing solutions require vast amounts of annotated data for training, which makes them difficult to apply to new domains due to the lack of annotations. In addition, a number of research studies concentrate more on the quality of extraction than on the scalability and efficiency of the process.

Another important challenge faced by existing approaches is the extraction of knowledge from complex and long documents that contain nested entities, ambiguous relations, and long-range dependencies. While pipeline-based approaches can still be affected by error propagation, end-to-end approaches may require huge computational resources and extensive training.

Survey articles indicate various challenges faced by existing approaches, such as graph completion, reasoning, knowledge fusion, multilingual knowledge graph construction, and heterogeneous data source integration. Moreover, only a few research works have tested their approaches on real-life applications with different types of documents. Hence, there is a clear need for a scalable and automated system for knowledge graph construction.

Chapter 3

Methodology

3.1 Overview and System Architecture

The proposed system follows a systematic approach to convert unstructured documents into a structured knowledge graph using Natural Language Processing (NLP) and the Neo4j graph database. The methodology consists of several stages, beginning with document ingestion and text extraction, followed by text preprocessing, Named Entity Recognition (NER), triple generation, graph construction in Neo4j, and finally, visualization of the generated knowledge graph using PyVis. Each stage was implemented and integrated to ensure accurate extraction and representation of knowledge from textual documents.

3.1.1 Algorithm for knowledge graph generation

Input: PDF, DOCX or TXT document

Output: Interactive knowledge graph

1. Accept the uploaded document through the Streamlit interface.
2. Validate the document format (PDF, DOCX, TXT).
3. Store the uploaded file temporarily.
4. Extract textual content from the document.
5. Pre-process the extracted text.
6. Perform Named Entity Recognition (NER) using the spaCy NLP model
7. Extract relationships between entities using dependency parsing and custom rule based extraction.
8. Generate Subject–Relation–Object (SRO) triples.
9. Remove duplicate triples.
10. Store the generated triples in the Neo4j Graph Database.
11. Retrieve graph statistics from Neo4j.
12. Generate an interactive knowledge graph using PyVis.
13. Display the generated graph and extracted information in the Streamlit application.

System Architecture

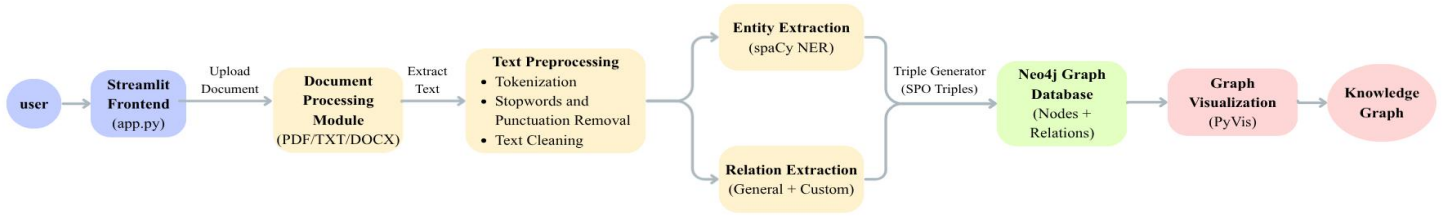


Figure 3.1: System Architecture

3.2 Document Ingestion Module

Document ingestion is the process of accepting, identifying, and reading documents so that their contents can be processed by the system. Its function is to accept unstructured documents from users, extract the textual content from them, and convert the data into a format suitable for Natural Language Processing (NLP) tasks. Without this module, the NLP system is unable to process the files, and Neo4j receives no data. Hence, the knowledge graph cannot be built.

The module is implemented using the Streamlit framework, which provides a simple and interactive file upload interface. After a document is selected, the uploaded file is temporarily stored on the local system and then forwarded for text extraction.

3.2.1 Supported Document Formats and Libraries used

To allow users to work with various kinds of unstructured data, the system supports three commonly used document formats: PDF, DOCX, and TXT. Users can easily upload and process research papers, journals, reports, news articles, and other text documents. The system does not require users to perform any manual file conversion. It works with different document sources since it supports multiple file formats.

Different document types require different Python libraries to process their respective file formats. The main library used to extract text from PDF documents is **pdfplumber**, which provides accurate text extraction. Some PDF documents that **pdfplumber** cannot process are handled using the **PyPDF2** library. For Microsoft Word documents, text extraction is performed using the **python-docx** library. Plain text files (.txt) are read using Python's built-in file functions. This combination of libraries provides the system with the accuracy and reliability required to extract text from documents in the supported file formats.

3.2.2 File Upload Process

To build the document upload process, the `st.file_uploader()` component from Streamlit is used. This creates an interface where users can select files for upload. Users can upload documents through the application's upload button. The system performs an

extension check on the selected file to validate it, accepting only files in the supported formats: PDF, DOCX, and TXT. The process continues only after a valid document has been uploaded. The type parameter in the upload component restricts the files that can be uploaded to the supported document formats and ensures that only documents compatible with the application are accepted for text extraction and analysis.

3.2.3 Temporary File Handling

After the document is uploaded, it is stored temporarily using Python's `tempfile` module. Temporary storage is required since most document processing libraries operate on file paths rather than uploaded file objects. Therefore, the uploaded document is written to a temporary location while preserving its original file extension. Once the text extraction process is completed, the temporary file is no longer required. This approach simplifies document processing while avoiding the need to permanently store user files on the system.

3.2.4 Text Extraction

After the document is successfully uploaded, it is extracted and converted into plain text for processing. The text extraction process is performed by the `extract_text_document()` function, which automatically identifies the uploaded file type and uses the appropriate extraction method based on the Python libraries mentioned above.

Once the text has been extracted, it is displayed in the **Extracted Text Preview**, as shown in **Figure 3.2**. This allows the user to verify that the document has been read correctly before proceeding to the next stage. If the uploaded document contains no readable text or the extraction fails, the application displays an error message to notify the user.



Figure 3.2: Extracted Text Preview

The extracted text serves as the input for the **Text Preprocessing** module, where unnecessary characters and irrelevant words are removed before performing entity and relationship extraction.

3.3 Text Preprocessing

3.3.1 Natural Language Preprocessing

Natural Language Processing (NLP) represents a branch of Artificial Intelligence aimed at providing computers with the ability to understand, process, and analyse human language. Within the described system, the **spaCy** NLP library is used to perform text preprocessing, named entity recognition, and relation extraction due to its fast processing, ease of use, and beginner-friendly nature.

Pre-trained models within the **spaCy** library already possess an understanding of English grammar, sentence structure, and entities such as people, organizations, dates, and numbers, making the system capable of efficient and accurate processing of textual data without requiring a custom-trained NLP model.

Typically, raw textual data is unstructured and noisy, containing typos, punctuation inconsistencies, stop words, and redundant information. Text preprocessing helps convert such data into a structured form and makes it possible to extract useful features from it.

3.3.2 Tokenization and Sentence Segmentation

Computers cannot understand paragraphs and sentences the way humans do. Instead, they break paragraphs into sentences and then further divide the text into smaller pieces known as **tokens**. For example, as shown in **Figure 3.3**, humans can easily interpret the sentence "Steve Jobs founded Apple." However, a computer processes it as the sequence ["Steve", "Jobs", "founded", "Apple"].

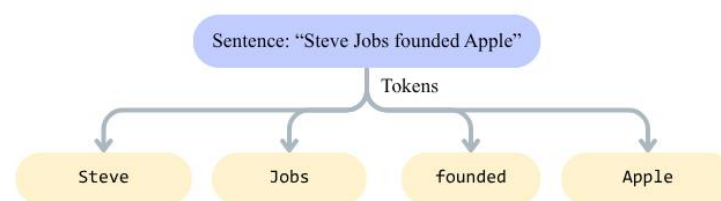


Figure 3.3: Tokenization of Input Sentence

This process is known as Tokenization which allows the NLP to analyse each word individually before identifying it as entity and relation.

3.3.3 Stop word,Punctuation and Ignore Word Removal

After tokenization, the extracted text is further refined by removing stop words, punctuation marks, and predefined ignore words. Stop words are commonly occurring words such as *is*, *a*, *an*, *the*, etc., which generally do not contribute meaningful information for entity or relationship recognition. Similarly, punctuation marks are removed as they do not provide any value during Natural Language Processing (NLP).

In addition to stop words, the system also removes a predefined list of ignore words that frequently appear but do not contribute to knowledge graph generation, such as

section, method, approach, and paper. Removing these words reduces noise and helps prevent the creation of unnecessary entities.

The uploaded text is first processed using the **spaCy** language model. Each token is checked to determine whether it is a stop word, a punctuation mark, or an ignore word. Only the significant tokens are retained and combined to produce clean text, which is then sent to the Named Entity Recognition (NER) module for further processing.



Figure 3.4: Processed Text

The text is pre-processed and is free from stop words, punctuation marks, and ignore words, providing cleaner input for the Named Entity Recognition (NER) module and improving the accuracy of entity and relationship extraction.

3.4 Named Entity Recognition

Named Entity Recognition (NER) is used to identify and classify important information, known as **entities**, from unstructured text into predefined categories such as **Person, Organization, Date, Time, Location, and Money**. Instead of treating every word equally, NER enables the system to recognize words that represent meaningful real-world objects.

NER is used to automatically identify significant entities from uploaded documents such as research papers, reports, newspapers, and articles. These extracted entities form **nodes** in the knowledge graph and are used to identify relationships between different pieces of information.

3.4.1 Entity Categories

The **spaCy** NLP library's pre-trained **en_core_web_md** model is used for Named Entity Recognition because it is fast and accurate. It can identify different categories of entities present in the text.

The system mainly utilizes the entity types that are most relevant for knowledge graph construction.

Table 3.1: Common entity types recognized by the spaCy NLP model

Entity Label	Description	Example
PERSON	Name of an individual	Steve Jobs
ORG	Organization	Apple
GPE	Country, State or City	India, London
DATE	Calendar dates	2026, June 2025
MONEY	Monetary values	\$500

3.4.2 Entity Extraction Process

When the text is pre-processed, it is passed to the spaCy Named Entity Recognition (NER) model. The model analyses every sentence and identifies important entities present in the document. Every detected entity is assigned an appropriate entity label such as PERSON, ORG, GPE, DATE, or MONEY.

Figure 3.5 illustrates the overall entity extraction process. The input sentence is analysed using spaCy, which identifies and classifies the important entities into their respective categories.

For example, in the sentence:

"Mayank Rathee is the founder of SpaceTax."

The NER model extracts the following entities:

- Mayank Rathee → PERSON
- SpaceTax → ORG

The extracted entities are stored along with their corresponding labels and sent for further processing.

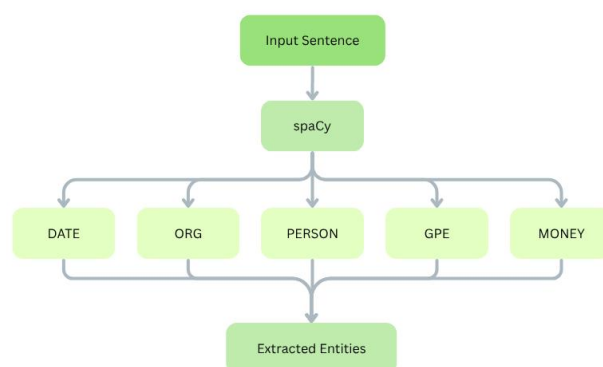


Figure 3.5: Entity Extraction Process using spaCy NER

3.4.3 Duplicate Removal

During entity extraction, the same entity may appear multiple times within a document. Storing duplicate entities would increase redundancy and unnecessarily enlarge the graph. To solve this issue, all extracted entities are compared, and repeated entities are removed. This produces a cleaner dataset and improves the efficiency of the generated knowledge graph.

3.4.4 Entity Extraction

The extracted entities are stored along with their corresponding entity labels. These entities serve as input to the next step of the system, where semantic relationships between the identified entities are extracted. The generated entity list is then forwarded to the Relationship Extraction module for the generation of Subject–Relation–Object (SRO) triples.

The implementation and output of the entity extraction process are presented and discussed in the Results and Discussion chapter using sample documents.

3.5 Relationship Extraction

Relationship Extraction is the process of identifying meaningful connections between entities present in a sentence. After Named Entity Recognition (NER), the system determines how these entities are related to each other. The extracted relationships are represented in the form of triples, also known as Subject–Relation–Object (SRO) triples.

For example, consider the sentence:

"Mayank founded SpaceTax in 1901."

The extracted triple is:

- (Mayank, FOUNDED,SpaceTax)
- (SpaceTax, FOUNDED_IN, 1901)

Representing information in triple format makes it easier to store the extracted knowledge in the Neo4j Graph Database and visualize it as interconnected nodes and relationships.

3.5.1 Dependency Parsing Using spaCy

The proposed system uses spaCy's dependency parser to identify the grammatical structure of each sentence. The parser analyses the relationships between words and determines:

- Subject (who performs the action)

- Root Verb (the relationship)
- Object (who or what receives the action)

For every sentence, the system identifies:

- Subject using dependency labels `asnsbj` and `nsbjpass`.
- Relation using sentence's root verb
- Object using dependency labels such as `asobj`, `pobj` and `attr`.

The extracted verb is converted into uppercase and cleaned before being used as the relationship name in generated triple.

3.5.2 Custom Relationship Extraction

Although dependency parsing extracts many relationships automatically, research papers often contain domain-specific expressions that cannot always be identified correctly.

To improve extraction accuracy, the system implements a set of rule-based custom relationships. Each sentence is analysed for predefined patterns and keywords. Whenever a matching pattern is detected, the corresponding relationship is generated. The combination of dependency parsing and rule-based extraction improves the overall quality of the generated relationships.

3.5.3 Triple Generation

After both the dependency-based and custom relationship extraction modules complete their execution, the extracted relationships are combined into a single collection. Duplicate triples are removed to improve the quality of the graph, and the output is stored as a list of unique Subject–Relation–Object (SRO) triples.

Figure 3.6 shows the triple generation process. The identified subject, relation, and object are combined to form a structured SRO triple. This structured representation converts unstructured textual information into machine-readable knowledge.

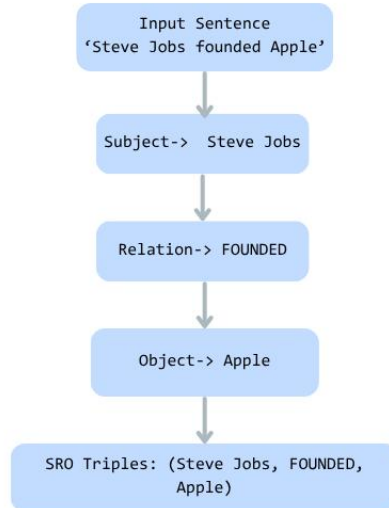


Figure 3.6: Subject–Relation–Object (SRO) Triple Generation Process

3.6 Neo4j Graph Database and Knowledge Graph Visualization

3.6.1 Neo4j Graph Database

After the steps involved in entity recognition and relation extraction have been performed, the SRO triples thus formed are stored in the database. While relational databases store data in tabular form, Neo4j graph database stores data in the form of nodes, relationships, and properties.

In the proposed system:

For instance, the extracted triple: (Steve Jobs, FOUNDED, Apple) is represented in Neo4j as:

Steve Jobs—FOUNDED—► Apple

With a graph-based structure representing and querying relationships of different entities is straightforward.

3.6.2 Storing Triples in Neo4j

The generated triples are automatically added into Neo4j using the official Neo4j Graph Driver Python Driver. Each subject and object is created as an Entity node using the MERGE command which creates the entity node if it does not exist in the Graph Database and prevents the duplication of the entity node.

Similarly, relationships are created between the subject and object nodes, which enables the connected entities to create a knowledge graph. When nodes are added to the Graph, the application queries the Graph Database to get number of nodes and relationships. This provides the size of the generated Graph.

3.6.3 Knowledge Graph Visualization using PyVis

Although Neo4j stores the graph efficiently, an interactive visualization helps to better understanding of the extracted data. Thus, the proposed system uses the PyVis library to build an interactive knowledge graph.

The generated graph supports zoom, drag, pan and automatic node arrangement which makes it easier to explore the extracted knowledge.

3.6.4 Streamlit Integration

The generated PyVis graph is saved as an HTML file and embedded directly into the Streamlit application. This feature enables the user to upload a document, extract entities and relationships, create a knowledge graph, and visualize the results within a single interface without the need for any additional software.

The application also displays graph statistics, including the total number of extracted entities, generated relationships, graph nodes, and graph edges, providing users with a complete overview of the generated knowledge graph.

Chapter 4

Results and Analysis

4.1 Software Configuration

The Automated Knowledge Graph Builder has been designed and developed using Python and a variety of open-source libraries. Python was selected due to its simplicity, flexibility, and suitability for web application development. Streamlit was used for the graphical user interface, while Neo4j was used as the graph database to store entities and relationships in the form of a knowledge graph. Each component was selected based on its functionality and role in the knowledge graph generation process.

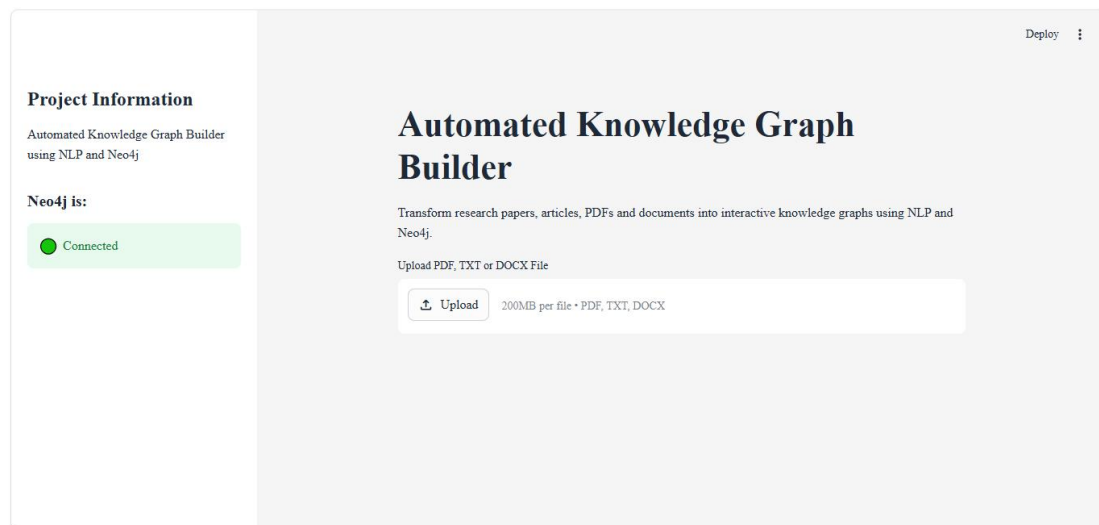
Table 4.1: Software Configuration

Library	Purpose
Python	Core Programming Language
Streamlit	Development of the web-based user interface.
spaCy(en_core_web_md)	Text pre-processing, Named Entity Recognition and dependency parsing.
Neo4j Graph Database	Storage of entities and relationships as a graph data.
Neo4j Python Driver	Establish connection between Python and Neo4j.
PyVis	Interactive visualization of generated knowledge graph.
Pandas	Displaying extracted entities and relationships in a tabular format.
tempfile	Temporary storage of uploaded documents during processing.
pdfplumber	Text extraction from PDF documents.
PyPDF2	Backup library for reading PDF files.
python-docx	To read and extract text from docx.

Built-In file handling	Used for reading plaintext documents.
------------------------	---------------------------------------

4.2 Application Interface

The Streamlit framework was used to create a friendly and interactive system. Users can completely control text extraction and preprocessing methods. Furthermore, they can use the system to create a knowledge graph by performing named entity recognition, relationship extraction, and graph visualization. An example of the application interface is shown in **Figure 4.1**.



(Figure 4.1:StreamlitApplication Main Page)

The application also includes options to enable the users to inspect the output at each iteration of the pipeline. Some of these options are the preview of the extracted text, the display of the processed text, the visualization of the extracted entities, the relationship extraction results, the statistics of the graph, and the interactive knowledge graph. These options enable the users to validate the extracted information and comprehend the knowledge retrieved from the document. The next sections show the evaluation experiments and the operation of the system with several test case documents.

4.3 Evaluation using Different Documents

The developed system was evaluated using three different types of unstructured documents to test the performance of the system. The selected datasets include a research paper, a company annual report and an online article as shown in **Table 4.2**. These documents differ in writing style, document structure and content allowing the system to be tested under different real-world scenarios.

Table 4.2: Test Documents used for Results

Test Document	Format	Type	Purpose
Microsoft Annual Report 2024	DOCX	Annual Report	Corporate Document
Understanding AI Technology	PDF	Research Paper	Technical Document
The uncontroversial ‘thingness’ of AI	TXT	Article	General text

4.3.1 Pre-processing Results

After the document is uploaded, the text is pre-processed before being sent to NER. During this stage, the text is cleaned, as shown in **Figure 4.2 (a) - 4.2 (c)**. Stop words, punctuation or unnecessary words are removed while meaningful words are retained.

Processed Text

Dear shareholders colleagues customers partners
Fiscal year 2024 pivotal year Microsoft entered 50th year company second year AI platform shift milestones found reflecting Microsoft remained consequential company decade decade industry franchise value realize time time tech paradigms shifted seized opportunity reinvent stay relevant customers partners employees today
Microsoft platform tools company start founded 1975 belief creating technology enable create nearly 50 years later belief remains heart mission empower person organization planet achieve year moved talking AI helping customers translate real outcomes person organization institution

Figure 4.2(a): Pre-processed Text of Annual Report

Processed Text

Understanding AI Technology
concise practical readable overview Artificial Intelligence
Machine Learning technology designed non technical
managers officers executives
April 2020
Greg Allen Chief Strategy Communications
Joint Artificial Intelligence Center JAIC

Figure 4.2(b): Pre-processed Text of Research Paper

Processed Text

Abstract
commentary starts question AI come figured uncontroversially thing controversies engender Addressing question takes knowledge practices philosopher science Helen Verran named hardening categories processes characterise epistemology AI central constituent techniques technologies context stabilization AI enables investments associated techniques technologies AI status controversial works reiterate ontological status agency follows interventions field AI controversies fail trouble destabilise AI risk contributing uncontroversial reproduction deny

Figure 4.2(c): Pre-processed Text of Article

4.3.2: Entity Extraction Results

Subsequently, the entities were extracted from the pre-processed text. The extracted entities varied according to the content of each document demonstrating that the proposed system can process different document formats successfully.

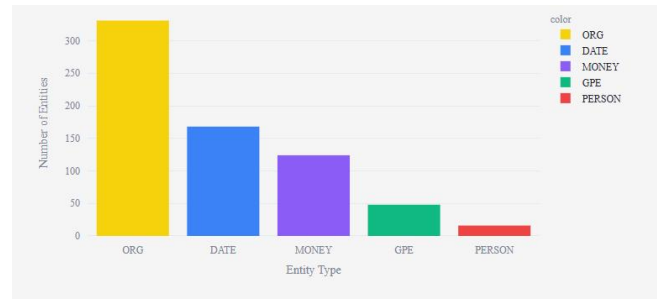


Figure 4.3 (a): Entity distribution of Annual Report



Figure 4.3 (b): Entity distribution of Research paper



Figure 4.3 (c): Entity distribution of Article

As shown in Figures 4.3(a)–(c), the list of identified entities depends on the type of document. The research papers and annual reports contain a large number of ORG and DATE entities due to their nature, containing many references to organizations, dates, financial years, institutes, and related information. In contrast to the previous documents, the news article contains more PERSON and ORG entities. It can be concluded that the NER model successfully identifies context-dependent entities across various types of documents. Table 4.3 shows the number of entities identified and the most frequent entity type in each test document.

Table 4.3: Entities Extracted by the system

Test Document	Number of Entities Extracted	Most frequent entity type
Microsoft Annual Report 2024	687	ORG, DATE
Understanding AI Technology	159	ORG, DATE
The uncontroversial ‘thingness’ of AI	54	PERSON, ORG

4.3.4 Relationship Extraction Results

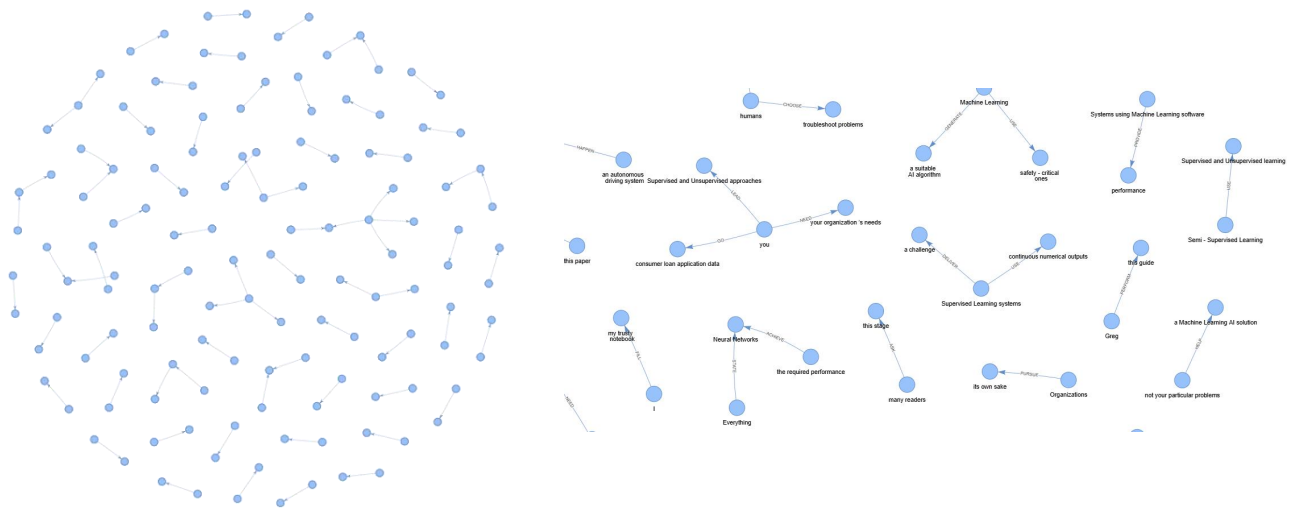
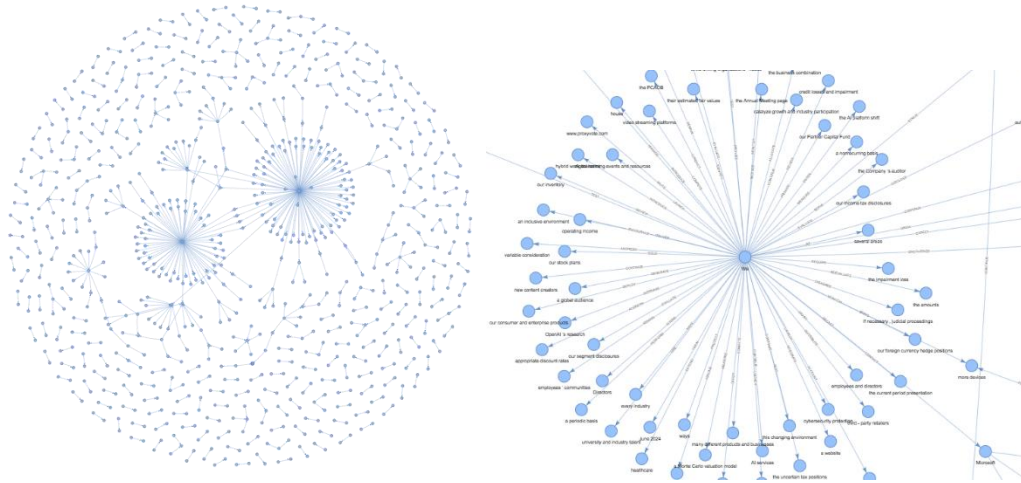
The number and type of extracted relationships varied according to the content of each test document. The Microsoft Annual Report produce the highest number of relationships because it contains detailed information about organizations, executives, financial data and corporate activities. The research paper generated relationships mainly between organizations and technical concepts while the article produced comparatively fewer relationships due to its shorter length and simpler content.

Table 4.4: Relationships Extracted by the system

Test Document	Relationships Extracted
Microsoft Annual Report 2024	623
Understanding AI Technology	74
The uncontroversial ‘thingness’ of AI	15

4.3.5 Knowledge Graph Generation Results

The SRO triples that were extracted were successfully saved in the Neo4j graph database to create the knowledge graph. The entities are represented by nodes,whereast he relationships extracted are represented by the edges connecting those nodes in the graph. This way of representing a graph allows for structured visualization of the extracted information from the uploaded documents.



Figures 4.4(a)–4.4(e) present the knowledge graphs generated from the three test documents. For the research paper and Microsoft Annual Report, both the complete graph and a zoomed-in view are presented. The complete view shows all entities within the overall structure of the graph, while the zoomed-in view focuses on the captured entities and their relationships. The Microsoft Annual Report produced the largest and most interconnected graph due to the presence of numerous organizations, financial information, and corporate relationships. The research paper generated a graph of moderate density with respect to the relationships among technical entities, while the news article produced a smaller graph with a number of independent connected components since it contained short and relatively independent statements.

The results demonstrate that the system successfully constructs knowledge graphs through the automatic extraction of entities and relationships and the storage of generated triples in the Neo4j Graph Database using unstructured documents. The system also allows users to explore the knowledge captured in the graphs in an intuitive manner through interactive visualization.

Table 4.5: Comparison of Knowledge Graph Generation Results

Test Document	Observation	Graph Generation Time (s)
Microsoft Annual Report 2024	Largest and highly connected graph	35.01
Understanding AI Technology	Moderately connected graph	8.25
The uncontroversial ‘thingness’ of AI	Smaller graph with multiple connected components	0.99

The graph generation time increased with the size and complexity of the input document. The Microsoft Annual Report required the highest processing time because it contained the largest number of extracted entities and relationships, whereas the text article required less than one second due to its smaller size.

The experimental results indicate that the Automated Knowledge Graph Builder creates structured knowledge graphs from unstructured documents with the aid of Natural Language Processing (NLP) and Neo4j. The builder was evaluated using a research paper, a Microsoft Annual Report, and a text article. In all cases, the application was able to extract the text, identify the named entities, create relationships, and build a knowledge graph. The Microsoft Annual Report contained large amounts of organizational, financial, and corporate relationship data, producing the largest and most comprehensive graph. The research paper produced a moderately dense graph with entities mainly of a technical nature, while the text article produced a smaller graph with fewer relationships due to its shorter length and the relatively independent nature of its statements. Although the system was able to build

knowledge graphs from different types of documents, some issues were encountered. Text extraction and, consequently, relationship extraction were incomplete or inaccurate for documents with complex layouts (e.g., research papers with multiple columns). The rule-based approach for relationship extraction also did not capture all relationships. These issues are discussed in the **Issues and Challenges** section. Overall, the experimental results demonstrate that the proposed solution is able to automatically convert unstructured documents into structured knowledge graphs with significantly less manual effort while allowing users to visualize the relationships among different entities in a user-friendly manner.

Chapter 5

Applications

The Automated Knowledge Graph Builder Using Natural Language Processing (NLP) has numerous applications in diverse fields that involve processing huge amounts of unstructured textual data. The main applications include:

5.1 Automated Knowledge Graph Generation

The system automatically converts unstructured documents such as research papers, reports, and articles into an interactive knowledge graph without requiring manual annotation. This reduces the effort required to organize and visualize textual information.

5.2 Academic and Research Institutions

The system assists researchers and students in analyzing research papers, journals, and technical documents. It enables the automatic extraction of concepts and relationships, making the process of literature analysis easier.

5.3 Knowledge Management

It allows organizations to transform reports, manuals, and policies into knowledge graphs. This facilitates information organization, access, and knowledge sharing.

5.4 Semantic Search Engines

Knowledge graphs improve the functioning of search engines by enabling not only keyword-based searches but also an understanding of the semantic relationships among entities.

5.5 Business Intelligence

Organizations can use customer reports, market research documents, and organizational records to analyse and extract relationships, aiding strategic decision-making.

5.6 Healthcare

Medical papers, patient records, and clinical reports can be analyzed to identify relationships among diseases, symptoms, medicines, and treatments, assisting healthcare professionals in knowledge discovery.

5.7 Document Analytics

The solution helps organizations analyse large collections of documents through entity extraction, relationship identification, and visualization of the extracted knowledge.

Chapter 6

Issues and Challenges

Although the proposed Automated Knowledge Graph Builder successfully extracts entities and relationships from multiple document formats and generates an interactive knowledge graph. Multiple challenges were encountered during the development and testing of application.

6.1 Processing of Research Papers

Research papers contain complex sentence structures, technical terminology, citations, references, figure captions and multi-column layouts. These characteristics make entity and relationship extraction more difficult. As a result, the generated knowledge graph may contain irrelevant or incomplete nodes.

6.2 PDF Text Extraction

PDF documents are created using different fonts, layouts and encoding standards. During text extraction, some documents may lose formatting, merge words together or produce unreadable characters which can affect the quality of the extracted information.

6.3 Rule-Based Relationship Extraction

The relationship extraction module is based on predefined linguistic rules and sentence patterns. While this works for identifying many common relationships such as `FOUNDED`, `CEO_OF` and `HEADQUARTERED_IN`, this approach may not work for extraction of many complex relationships which may be expressed in many different ways

6.4 Entity Recognition Limitations

The system uses the pre-trained spaCy NLP model for Named Entity Recognition. Although the model provides good accuracy for general English text, it may not correctly recognize domain-specific terms, abbreviations or newly emerging entities commonly found in research articles.

6.5 Graph Quality

The quality of a generated knowledge graph can be attributed to the quality of the entities and relations that were extracted. Poor extraction can cause the nodes to be noisy, relationships to be redundant and graph components to be disconnected. All of this impacts the readability of the graph.

6.6 Scalability

As it currently stands, the system is only meant to work on a single document at a time. If multiple, or if extremely large, documents are processed at once, this can lead to a performance burden to the system because of increased time and resource consumption.

Chapter 7

Conclusion

This project outlined the design and development of an Automated Knowledge Graph Builder, incorporating Natural Language Processing (NLP). The system developed constructs knowledge graphs automatically from unstructured documents. This is performed by extracting the documents, undergoing preliminary text processing, recognizing named entities, identifying relations, and saving the constructed Subject-Relation-Object triples in the Neo4j database. The system designed in this project was tested using three distinctly different types of documents, which include a sample research paper, a sample Microsoft Annual Report, and a sample text article. The results of the sample experiments proved successful processing of the documents in the PDF, DOCX, and TXT formats and generation of significant knowledge graphs from each of the documents. Document processing was made user-friendly by Streamlit. Simplicity and interactivity were brought by Streamlit while efficient storage and visualization of the knowledge extracted was made possible by Neo4j and PyVis, respectively. Processing documents that have a complex structure and an advanced degree of unstructured content posed some challenges. The system, however, performed satisfactorily. Knowledge graphs constructed by the system expressed the relations of the entities extracted and significantly reduced the amount of time and effort involved in the analysis of large text documents. The project accomplished the task of converting unstructured text to structured knowledge graphs. This project is an excellent example of practical implementation of the combination of NLP, graph databases, and knowledge extraction. Furthermore, this project provides a great starting point for innovations in automated knowledge graph generation.

Chapter 8

Future Scope

The system extracts entities and relations from unstructured text to build knowledge graphs using Natural Language Processing. However, there are some improvements that can enhance the project for the future. Currently, only PDF, DOCX and TXT files can be processed. In the future, more file types can be added with the inclusion of document formats like HTML, XML and scanned documents using Optical Character Recognition. Thus far, the extraction of relationships relies on dependency parsing along with a few set patterns built using rules. The substitution of these rules with advanced AI/ML algorithms can improve the ability to understand relations and subsequently decrease the number of erroneous extractions. Knowledge graph representation also draws a lot of improvement opportunities. The ability to search, filter, and group nodes as well as improved layouts for the graphs would enable users to navigate and interact with a knowledge graph of a much larger scale. Integration with a chatbot or question-answering system would enable users to ask questions in a natural way and receive responses from the knowledge graph built in the system. Finally, the system would be an online application where users could build personal accounts with cloud database functionality. The hosting of the application on the cloud would enable all users to build knowledge graphs with their documents and retrieve their projects from anywhere.

References

- [1] A. Hogan, E. Blomqvist, M. Cochez, C. D'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. Labra Gayo, R. Navigli, S. Neumaier, A.-C. Ngonga Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, and A. Zimmermann, "Knowledge Graphs," *ACM Computing Surveys*, vol. 54, no. 4, Art. 71, pp. 1–37, Jun. 2021, doi: 10.1145/3447772.
- [2] P. Schneider, T. Schopf, J. Vladika, M. Galkin, E. Simperl, and F. Matthes, "A Decade of Knowledge Graphs in Natural Language Processing: A Survey," in *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 601–614, 2022.
- [3] R. M. Bakker and D. L. Di Scala, "From Text to Knowledge Graph: Comparing Relation Extraction Methods in a Practical Context," in *CEUR Workshop Proceedings*, 2024.
- [4] H. Yu, H. Li, D. Mao, and Q. Cai, "A Relationship Extraction Method for Domain Knowledge Graph Construction," *World Wide Web*, vol. 23, no. 2, pp. 735–753, 2020.
- [5] P.-L. Huguet Cabot and R. Navigli, "REBEL: Relation Extraction by End-to-end Language Generation," in *Findings of the Association for Computational Linguistics: EMNLP 2021*, pp. 2370–2381, 2021.
- [6] I. Melnyk, P. Dognin, and P. Das, "Knowledge Graph Generation From Text," in *Findings of the Association for Computational Linguistics: EMNLP 2022*, pp. 1610–1622, 2022.
- [7] D. Fensel, U. Şimşek, K. Angele, E. Huaman, E. Kärle, O. Panasiuk, I. Toma, J. Umbrich, and A. Wahler, *Knowledge Graphs*. Cham, Switzerland: Springer, 2020, ch. 1, "Introduction: What Is a Knowledge Graph?"
- [8] R. Fan, L. Wang, J. Yan, W. Song, Y. Zhu, and X. Chen, "Deep Learning-Based Named Entity Recognition and Knowledge Graph Construction for Geological Hazards," *ISPRS International Journal of Geo-Information*, vol. 9, no. 1, Art. no. 15, 2020.
- [9] M. Y. Jaradeh, K. Singh, M. Stocker, A. Both, and S. Auer, "Information Extraction Pipelines for Knowledge Graphs," *Knowledge and Information Systems*, vol. 65, no. 5, pp. 1989–2016, 2023.
- [10] C. Peng, F. Xia, M. Naseriparsa, and F. Osborne, "Knowledge Graphs: Opportunities and Challenges," *Artificial Intelligence Review*, vol. 56, pp. 13071–13102, 2023.
- [11] Z. Chen, Y. Zheng, J. Ge, W. Yu, and Z. Wang, "A Parallel Model for Jointly Extracting Entities and Relations," *Neural Processing Letters*, vol. 56, Art. no. 165, 2024.